TrendVibe Atelier — Complete Project Interview Master Guide

Role: Java Full Stack Developer • **Architecture:** Jakarta EE 10 MVC, Tomcat 10.1, JDBC, MySQL, HTML5/CSS3/JS • **Live:** indranil735.github.io/trendvibe-ecommerce/

1. The 60-Second Project Elevator Pitch (Say This First in the Interview)

"TrendVibe Atelier is an enterprise-grade luxury fashion and beauty e-commerce web platform inspired by Myntra, Ajio, and Nykaa. I built it using a clean Java MVC (Model-View-Controller) architecture running on Apache Tomcat 10.1 with Jakarta EE 10 and Java 21. The backend is powered by Servlets handling HTTP routing, JSP with Jakarta Standard Tag Library (JSTL) for dynamic views, and a decoupled Data Access Object (DAO) pattern communicating with MySQL via JDBC connection pooling and PreparedStatements for SQL injection defense. On the front end, it features a responsive, mobile-first design system with product filtering, cart management, real-time coupons, multi-step checkout, and order tracking."

2. Architectural Flow & Pin-to-Pin Request Lifecycle

When an interviewer asks: **'Walk me through the flow of a request in your project'**, explain this exact 5-layer pipeline:

Step / Layer	Component in TrendVibe	What Actually Happens
1. Client / Browser	HTML5 / CSS3 / JavaScript	User clicks 'Add to Bag' or 'Place Order'. An HTTP POST request is dispatched to /cart/add or /checkout/place-order.
2. Web Container	Apache Tomcat 10.1 + web.xml	Tomcat receives the HTTP request, checks URL mappings in web.xml, allocates a worker thread, and invokes the matching Servlet's service() method.
3. Controller	HttpServlet (e.g. CartController)	Extracts request parameters (productId, size), accesses HttpSession, performs business validation, and calls DAO methods.
4. Data Access (DAO)	DAO Pattern + DBConnection	CartDAO acquires a Connection from DBConnection, executes parameterized SQL via PreparedStatement, and maps ResultSet rows into Model POJOs.
5. View Rendering	JSP + JSTL + Expression Language	Servlet sets data in request.setAttribute() and forwards via RequestDispatcher.forward() to JSP, which generates HTML with JSTL loops and EL tags.

3. Pin-to-Pin Codebase Breakdown

Layer	Key Classes / Files	Role and Responsibilities in Code
Model Layer (POJOs) com.ecommerce.model	• User.java • Product.java • ProductSize.java • Category.java • Cart.java • CartItem.java • Order.java • OrderItem.java	Plain Old Java Objects encapsulating database entities with private fields, getters, setters, and constructors. Cart maintains total calculations, and ProductSize models inventory stock per size variant.
Database Layer com.ecommerce.dao	• DBConnection.java • MySQL Connector/J 8.3.0	Singleton/Factory pattern establishing JDBC connectivity using DriverManager.getConnection() with auto-reconnect, UTF-8 character encoding, and safe connection close routines.
DAO Interfaces & Implementations com.ecommerce.dao	• IUserDAO / UserDAO • IProductDAO / ProductDAO • ICategoryDAO / CategoryDAO • ICartDAO / CartDAO • IOrderDAO / OrderDAO	Decoupled interface contracts declaring CRUD operations. Implementations use PreparedStatement to execute SQL safely, handle transactions with commit()/rollback() on order checkout, and map results to Models.
Controller Layer (Servlets) com.ecommerce.controller	• ProductController (/home, /products) • CartController (/cart, /cart/add) • CheckoutController (/checkout) • LoginController (/login) • RegisterController (/register) • ProfileController (/profile) • LogoutController (/logout)	Extends jakarta.servlet.http.HttpServlet. Overrides doGet() and doPost() to parse parameters, enforce authentication guards via HttpSession, interact with DAOs, and forward to views.
View Layer (JSP & Webapp) src/main/webapp/WEB-INF/views/	• home.jsp • products.jsp • product-details.jsp • cart.jsp • checkout.jsp • orders.jsp • order-details.jsp • header.jsp & footer.jsp	Protected behind /WEB-INF/ so clients cannot access them directly without passing through a Servlet. Uses Jakarta JSTL (<c:forEach>, <c:if>, <c:choose>) and XML-safe Expression Language.
Configuration & Build Root project files	• web.xml (Deployment Descriptor) • pom.xml (Maven Build) • schema.sql & sample_data.sql	web.xml maps servlets, URL patterns, session timeouts, and welcome files. pom.xml compiles with Java 21 and bundles Jakarta Servlet 5.0, JSTL 2.0, and MySQL Connector 8.3.

4. Top 15 Real Interview Questions & Word-for-Word Answers

Q1: Why did you use Jakarta EE instead of javax.servlet?

With the transition of Java EE to the Eclipse Foundation, the namespace moved from javax.* to jakarta.* starting in Tomcat 10 and Jakarta EE 9/10. Because my project targets Apache Tomcat 10.1 and Java 21, I use jakarta.servlet.http.HttpServlet and jakarta.tags.core for JSTL to ensure 100% compliance with modern enterprise Java standards.

Q2: Why do you put your JSP files inside /WEB-INF/views/ instead of the root webapp folder?

Security best practice. Any file inside the /WEB-INF directory cannot be accessed directly by a client browser typing a URL (e.g. localhost:8080/cart.jsp will return 404). Access is strictly mediated by my Servlets via RequestDispatcher.forward(). This guarantees that users cannot bypass authentication, validation, or business logic.

Q3: Why did you create DAO Interfaces (like IUserDAO) instead of directly writing DAO classes?

It follows the Dependency Inversion Principle and Loose Coupling from SOLID principles. The Controller depends on the abstraction (interface), not the concrete implementation. If tomorrow we switch from MySQL to PostgreSQL or MongoDB, we only write a new DAO implementation class without modifying a single line of Controller code.

Q4: How did you prevent SQL Injection in your project?

I used PreparedStatement for all SQL queries instead of Statement with string concatenation. PreparedStatement pre-compiles the SQL template and sends parameters separately as data, ensuring that malicious inputs (e.g., ' OR '1'='1) are treated as literal strings and can never alter the query structure.

Q5: How does Session Management work in TrendVibe?

When a user logs in, LoginController calls request.getSession(true). This generates a unique JSESSIONID stored in an HTTP cookie on the client side. The server stores the User object and Cart in the session (session.setAttribute("user", user)). On subsequent requests, the JSESSIONID cookie verifies the user's active session without re-prompting for credentials until logout or session timeout.

Q6: How do you handle database transactions during checkout?

In OrderDAO.createOrder(), multiple database operations must succeed atomically: inserting the Order record, inserting each OrderItem, and updating product stock. I disable auto-commit using connection.setAutoCommit(false), execute all inserts, and call connection.commit(). If any step fails, catch block executes connection.rollback() to prevent orphaned order records.

Q7: Explain the Servlet Life Cycle in your application.

1. Loading & Instantiation: Tomcat loads the servlet class and creates an instance once. 2. Initialization: Tomcat calls init(ServletConfig) to load configuration. 3. Request Servicing: For every incoming HTTP request, Tomcat invokes service() on a pooled thread, which delegates to doGet() or doPost(). 4. Destruction: When the server shuts down, Tomcat calls destroy() to release resources.

Q8: What is the difference between RequestDispatcher.forward() and HttpServletResponse.sendRedirect()?

• forward() happens entirely on the server side: the browser URL remains the same, request/response objects and attributes are preserved, and only 1 round-trip occurs. Used when routing from a Servlet to a JSP view. • sendRedirect() sends an HTTP 302 redirect back to the client: the browser URL changes, a new HTTP request is made, and request attributes are discarded. Used after POST actions (like logout or checkout) to prevent duplicate form submissions.

Q9: How do you handle password security in User registration?

Passwords should never be stored in plain text. In the project, we hash passwords with salt before inserting into the users table using a cryptographic hashing algorithm (such as SHA-256 or BCrypt). During login, the incoming password is hashed with the same salt and compared against the stored hash.

Q10: What is the difference between doGet() and doPost()?

doGet() is idempotent and used for retrieving data (e.g., browsing products, viewing profile). Parameters are visible in the URL query string and are cached by browsers. doPost() is used for state-modifying actions (e.g., submitting credentials, adding to cart, placing orders). Parameters are sent in the HTTP request payload, allowing large and sensitive payloads.

Q11: Why did you use JSTL and Expression Language (EL) instead of Java scriptlets (<% %>)?

Scriptlets mix Java code with HTML, which violates separation of concerns, makes pages unreadable, and creates maintenance nightmares. JSTL and EL provide clean tag-based syntax (, ${product.name}) that allows frontend developers and designers to work on JSPs without Java syntax errors.

Q12: How do you handle errors like 404 or 500?

In web.xml, we configure declarations mapping HTTP error codes (404 Not Found, 500 Internal Server Error) to user-friendly JSP pages. In Java code, unexpected exceptions are logged and caught in try-catch blocks to prevent raw stack traces from exposing system internals to users.

Q13: What design patterns did you implement in this project?

1. MVC (Model-View-Controller) for architectural separation. 2. DAO (Data Access Object) Pattern for decoupling data access. 3. Singleton Pattern in DBConnection to centralize connection management. 4. Factory / Interface abstraction in DAO layers.

Q14: How does the Cart work for both logged-in and guest users?

Cart is stored in the HttpSession. When a guest user adds items, the cart persists in their active session. When the user logs in, their existing session cart is synchronized with the database using CartDAO, allowing seamless checkout without losing selected products.

Q15: If this application receives 100,000 concurrent users, what would you optimize?

1. Replace basic JDBC connections with HikariCP connection pooling to prevent database connection exhaustion. 2. Introduce Redis for caching high-traffic product catalog queries and session storage. 3. Deploy multiple Tomcat instances behind an Nginx load balancer. 4. Serve static assets (images, CSS, JS) via a Content Delivery Network (CDN) like AWS CloudFront.

5. Database Schema Quick Reference

Table	Key Columns	Relationships
users	user_id (PK), full_name, email, password_hash, phone, role	1-to-many with orders and cart
categories	category_id (PK), category_name, slug	1-to-many with products
products	product_id (PK), name, description, price, discount_pct, category_id, image_url	Foreign key to categories
product_sizes	size_id (PK), product_id (FK), size_label (S/M/L/XL), stock_qty	Foreign key to products
cart	cart_id (PK), user_id (FK), created_at	1-to-many with cart_items
cart_items	cart_item_id (PK), cart_id (FK), product_id (FK), size, quantity	Foreign keys to cart & products
orders	order_id (PK), order_number, user_id (FK), total_amount, status, address	1-to-many with order_items
order_items	order_item_id (PK), order_id (FK), product_id (FK), size, quantity, unit_price	Foreign keys to orders & products